H264 视频编码崩溃修复报告

文档信息

项目	内容
项目名称	WeChat 视频会议系统
问题编号	H264-CRASH-20260728
发现时间	2026-07-28 20:18
修复完成	2026-07-29 00:00
修复人员	开发团队
影响版本	H264 编码路径启用后的所有构建
严重级别	P0 — 阻断核心功能
构建环境	Qt 5.12.11 + MinGW 7.3.0 32-bit (Release)

目录

1. 问题概述
2. 系统架构与线程模型
3. 排查过程（分阶段详解）
4. 根本原因分析
5. 解决方案（含完整代码实现）
6. 修改文件清单与差异对比
7. 测试验证
8. 调试代码清理记录
9. 经验总结与最佳实践
10. 附录

一、问题概述

1.1 问题表现

启用 H264 视频编码后，客户端程序在成功编码并发送**第一帧**视频数据后立即崩溃退出。崩溃无错误对话框、无异常日志，程序被操作系统强制终止。

典型崩溃日志时序：

```
20:18:32: Starting WeChat.exe ...
[SDL] SDL_Init(SDL_INIT_AUDIO) succeeded
slot_loginCommit 被调用, tel: "14414414414"
[TcpClient::InitNet] connect OK, m_sock=1832
slot_dealLoginRS
[LoginRS] AV connection status: audio_connected= true video_connected= true

=== H264编码器激活流程开始 ===
H264Encoder: using encoder libx264 (verified by avcodec_open2)
H264Encoder: init success, 640 x 480 @ 30 fps encoder: "libx264"
=== 视频启动完成, 编码模式: H264 ===

slot_sendVideoFrame called, img size: 640 x 480 , m_id: 3
H264Encoder::encode 640 x 480 frame: 0
[slot_sendVideoFrame] H264 encode: ba.size= 36944 totalFragments= 10

[slot_SendVideo] enter, packType= 10017 nlen= 4136
[TcpClient::SendData] m_sock=1832, nlen=4136, packType=10017
[slot_SendVideo] SendData OK, packType= 10017
...（10个分片全部发送成功）...
[slot_SendVideo] SendData OK, packType= 10017

20:18:52: 程序异常结束。
20:18:52: The process was ended forcefully.
20:18:52: WeChat.exe crashed.
```

1.2 影响范围

影响维度	说明
功能影响	H264 视频编码功能完全不可用，程序在第一帧后崩溃
用户体验	视频通话功能无法使用，程序意外退出
回退方案	仅能使用 JPEG 编码（画质差、带宽占用高约 5 倍、帧率低）
触发条件	启动视频通话，H264 编码器成功初始化后编码第一帧

1.3 关键特征

- 崩溃**总是在第一帧编码和发送完成后**发生
- 崩溃前所有操作（编码、分片、网络发送、内存释放）均报告成功
- 崩溃无 Windows 异常对话框，进程被直接强制终止

- 崩溃后无核心转储文件生成

二、系统架构与线程模型

2.1 视频处理全链路架构

理解崩溃问题需要先了解系统的多线程架构和数据流：



2.2 三线程职责说明

线程	类	职责	关键操作
Video线程	VideoRead	摄像头采集、格式转换、信号发射	cap.read() → cvtColor() → QImage → SIG_sendVideoFrame

线程	类	职责	关键操作
Main线程	ckernel / UserShow	事件循环、编码调度、UI绘制	slot_sendVideoFrame → encodeAndSendVideoFrame → paintEvent
Worker线程	ckernel::slot_SendVideo	网络发送、缓冲区释放	SendData() → delete[] buf

2.3 视频帧数据流（详细）

以下是单帧视频从采集到发送的完整数据流：

```
步骤1: Video线程 — 摄像头采集
  VideoRead::slot_getVideoFrame()
    ├── cap.read(frame)                    // OpenCV Mat (BGR格式)
    ├── cvtColor(frame, frame, BGR2RGB)    // 转为RGB
    ├── QImage image(frame.data, ...)      // ⚠️ 浅引用Mat数据
    ├── image = image.scaled(640, 480)     // 创建新QImage(独立数据)
    └── Q_EMIT SIG_sendVideoFrame(image)   // 排队到Main线程

步骤2: Main线程 — 接收与分发
  ckernel::slot_sendVideoFrame(QImage img)  // img是队列拷贝(浅拷贝)
    ├── slot_refreshVideo(m_id, img)        // 更新UI预览
    │     └── UserShow::slot_setImage(img)
    │           ├── m_img = img             // ⚠️ 浅拷贝(问题根源!)
    │           └── this->update()          // 触发paintEvent
    ├── QImage imgCopy = img.copy()         // 深拷贝用于编码
    └── QTimer::singleShot(0, [imgCopy]() {
            encodeAndSendVideoFrame(imgCopy)
        })

步骤3: Main线程 — H264编码
  ckernel::encodeAndSendVideoFrame(QImage img)
    ├── m_h264Encoder->encode(img)          // H264编码
    │     ├── img.copy()                    // 深拷贝
    │     ├── av_frame_make_writable()      // 确保帧可写
    │     ├── sws_scale()                   // RGB→YUV420P
    │     ├── avcodec_send_frame()          // 发送到编码器
    │     └── avcodec_receive_packet()      // 接收编码数据
    ├── 分片打包 (H264_FRAGMENT_SIZE=4096)
    └── Q_EMIT SIG_SendVideo(buf, nlen)     // 排队到Worker线程

步骤4: Worker线程 — 网络发送
  ckernel::slot_SendVideo(char* buf, int nlen)
    ├── m_pAVClient->SendData(0, buf, nlen) // TCP发送
    └── delete[] buf                        // 释放分片缓冲区
```

```
步骤5: Main线程 — UI绘制 (与步骤4并发!)
  UserShow::paintEvent()
    ├── m_img.scaled(...)                    // ⚠️ 读取像素数据(崩溃点!)
    ├── QPixmap::fromImage(m_img)
    └── painter.drawPixmap(...)
```

三、排查过程（分阶段详解）

3.1 阶段一：FFmpeg 帧缓冲区管理问题

3.1.1 假设

av_image_alloc 手动分配的帧缓冲区绕过了 FFmpeg 的 AVBufferRef 引用计数机制。avcodec_send_frame 内部调用 av_frame_ref 时发现 buf[] 为空，只能浅拷贝 data 指针。这导致编码器内部帧与外部帧共享同一块缓冲区，下一次 encode() 调用覆盖缓冲区时，编码器仍在读取 → 堆腐蚀。

3.1.2 验证方法

将 av_image_alloc 替换为标准的 av_frame_get_buffer，并添加 av_frame_make_writable 确保帧缓冲区可写：

```
// 修改前（有问题）
int bufSize = av_image_get_buffer_size(AV_PIX_FMT_YUV420P, width, height, 1);
m_yuvBuffer = (uint8_t*)av_malloc(bufSize);
av_image_fill_arrays(m_frame->data, m_frame->linesize, m_yuvBuffer,
                     AV_PIX_FMT_YUV420P, width, height, 1);

// 修改后（标准方式）
m_frame->format = AV_PIX_FMT_YUV420P;
m_frame->width  = width;
m_frame->height = height;
int ret = av_frame_get_buffer(m_frame, 0);  // 为每个平面创建 AVBufferRef

// 每次 encode 前调用
int ret = av_frame_make_writable(m_frame);  // 引用计数>1时自动分配新缓冲区
```

3.1.3 结果

日志显示 av_frame_get_buffer 成功（buf[0] 指针非空），编码和发送均成功，但**崩溃依旧**。

3.1.4 结论

帧缓冲区管理问题是真实存在的（需要修复），但不是崩溃的根本原因。排除此假设。

3.2 阶段二：FFmpeg DLL ABI 兼容性问题

3.2.1 假设

MSVC 编译的 FFmpeg DLL 与 MinGW 32-bit 存在 ABI 不兼容，导致 `avcodec_send_frame` / `avcodec_receive_packet` 内部堆腐蚀。

3.2.2 验证步骤

步骤 1：检查 FFmpeg DLL 的编译工具链

使用 Dependency Walker 或 `objdump` 检查 DLL 依赖：

```
ffmpeg\bin\avcodec-58.dll
  └── 依赖: msvcrt.dll (非 msvcrXXX.dll)
  └── 结论: MinGW 编译的 Zeranoe 构建，非 MSVC 编译
```

步骤 2：检查导入库格式

发现项目使用 MSVC 格式的 `.lib` 导入库，应使用 MinGW 格式的 `.dll.a`：

```
# 修改前（MSVC 格式，与 MinGW 链接器不完全兼容）
LIBS += $$PWD/ffmpeg-4.2.2/lib/libavcodec.lib \
        $$PWD/ffmpeg-4.2.2/lib/libavutil.lib \
        $$PWD/ffmpeg-4.2.2/lib/libswscale.lib

# 修改后（MinGW 标准格式）
LIBS += $$PWD/ffmpeg-4.2.2/lib/libavcodec.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavutil.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libswscale.dll.a
```

步骤 3：添加堆完整性检查

在 `H264Encoder::encode()` 中添加 `HeapValidate` 检查：

```
#include <windows.h>

// 编码前检查
if (!HeapValidate(GetProcessHeap(), 0, NULL)) {
    qCritical() << "HEAP CORRUPTED before avcodec_send_frame!";
```

```
    return QByteArray();
}

// 编码后检查
if (!HeapValidate(GetProcessHeap(), 0, NULL)) {
    qCritical() << "HEAP CORRUPTED after encoding!";
    return QByteArray();
}
```

3.2.3 结果

- 导入库已切换为 `.dll.a`
- `HeapValidate` **全部通过**，堆未腐蚀
- 崩溃依旧

3.2.4 结论

FFmpeg DLL 是 MinGW 编译的，与 MinGW 代码兼容。`.lib` → `.dll.a` 的切换是必要的（防止潜在问题），但不是崩溃原因。排除此假设。

3.3 阶段三：手动 RGB→YUV 转换缺陷

3.3.1 假设

手动 BT.601 RGB→YUV420P 转换代码可能存在缓冲区越界写入，腐蚀了相邻堆内存。

3.3.2 验证方法

用 FFmpeg 的 `sws_scale` 替代手动转换（DLL 确认为 MinGW 编译，`.dll.a` 链接正确）：

```
// 修改前（手动转换，可能越界）
for (int y = 0; y < m_height; y++) {
    for (int x = 0; x < m_width; x++) {
        uint8_t r = row[x * 3];
        // ... 手动计算 Y/U/V 分量
        yPlane[y * yStride + x] = yVal;
        if ((y & 1) == 0 && (x & 1) == 0) {
            uPlane[(y >> 1) * uStride + (x >> 1)] = uVal;
            vPlane[(y >> 1) * vStride + (x >> 1)] = vVal;
        }
    }
}

// 修改后（FFmpeg sws_scale，SIMD 优化、边界检查完善）
m_swsCtx = sws_getContext(width, height, AV_PIX_FMT_RGB24,
```

```
                            width, height, AV_PIX_FMT_YUV420P,
                            SWS_BILINEAR, nullptr, nullptr, nullptr);

const uint8_t *srcData[1] = { rgbImg.constBits() };
int srcLinesize[1] = { rgbImg.bytesPerLine() };
sws_scale(m_swsCtx, srcData, srcLinesize, 0, m_height,
          m_frame->data, m_frame->linesize);
```

3.3.3 结果

sws_scale 正常工作，HeapValidate 仍通过。崩溃依旧。

3.3.4 结论

手动转换无越界问题。sws_scale 替代是合理的（更可靠、更高效），但不是崩溃原因。排除此假设。

3.4 阶段四：精确定位崩溃位置

3.4.1 假设

崩溃不在编码路径中，而在编码完成后的 Qt 事件循环处理中。

3.4.2 验证方法 — 三重诊断插桩

插桩 1：paintEvent 全链路日志

在 UserShow::paintEvent 中添加分步日志：

```
void UserShow::paintEvent(QPaintEvent *event)
{
    qDebug() << "[paintEvent] enter, m_img.size:" << m_img.size()
             << "widget:" << this->width() << "x" << this->height();
    // ...
    qDebug() << "[paintEvent] scaling image...";
    QImage scaledImg = m_img.scaled(...);
    qDebug() << "[paintEvent] scaled OK:" << scaledImg.size();
    // ...
    qDebug() << "[paintEvent] done";
}
```

插桩 2：心跳定时器

在 ckernel 构造函数中添加 500ms 心跳定时器：

```
QTimer *heartbeat = new QTimer(this);
heartbeat->setInterval(500);
connect(heartbeat, &QTimer::timeout, []() {
    qDebug() << "[heartbeat] main thread alive";
});
heartbeat->start();
```

插桩 3：slot_SendVideo 的 delete[] 前后日志

```
qDebug() << "[slot_SendVideo] before delete[], buf=" << (void*)buf;
delete[] buf;
qDebug() << "[slot_SendVideo] after delete[], packType=" << packType;
```

3.4.3 关键发现

诊断日志输出显示：

```
[heartbeat] main thread alive                    ← 主线程正常运行
[heartbeat] main thread alive

slot_sendVideoFrame called, img size: 640 x 480  ← 第一帧到达
[paintEvent] enter, m_img.size: QSize(640, 480)  ← paintEvent 被调用
                                                   widget: 241 x 200
[slot_SendVideo] before delete[], buf= 0xb3abc80 ← Worker线程并发执行
[slot_SendVideo] after delete[], packType= 10017
[paintEvent] scaling image...                     ← Main线程进入 scaled()
[slot_SendVideo] before delete[], buf= 0xb3accb0 ← Worker线程继续
[slot_SendVideo] after delete[], packType= 10017
（没有 "scaled OK" 日志）                          ← ⚠️ 崩溃在 scaled() 内部!

23:35:45: 程序异常结束。
```

3.4.4 结论

崩溃发生在 `QImage::scaled()` 调用中，位于 `UserShow::paintEvent` 函数内。崩溃时：

- Main 线程正在执行 `m_img.scaled()` 读取像素数据
- Worker 线程正在并发执行 `delete[] buf` 堆操作
- Video 线程可能正在创建新的 QImage

四、根本原因分析

4.1 直接原因 — Qt 隐式共享的浅拷贝

UserShow::slot_setImage 中使用**浅拷贝**（Qt 隐式共享）赋值 m_img：

```
// 原代码（有 Bug）
void UserShow::slot_setImage(QImage &img)
{
    m_img = img;  // ⚠️ 浅拷贝！m_img 与 img 共享同一块像素数据
    m_lastTime = QTime::currentTime();
    this->update();
}
```

4.2 Qt 隐式共享机制详解

Qt 的 QImage 使用**隐式共享**（Implicit Sharing / Copy-On-Write）机制来减少内存拷贝开销：

```
QImage A(frame.data, 640, 480, RGB888);  // A 持有数据
QImage B = A;                             // 浅拷贝：A 和 B 共享同一块像素数据
                                          //         引用计数 refcount = 2

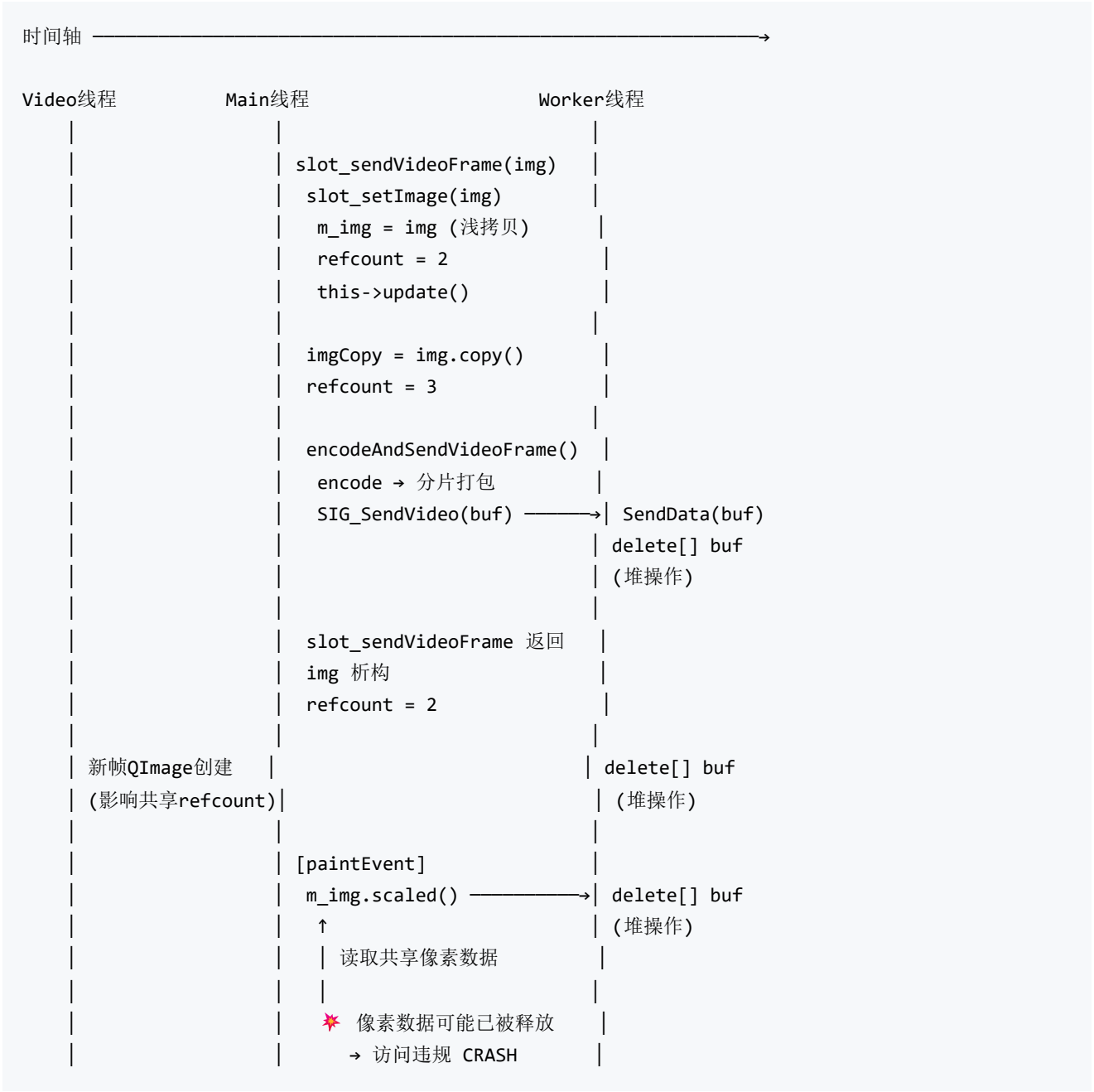
// 此时内存状态：
// ┌──────────────┐
// │ QImage A     │──────┐
// │ d-pointer ───┤      │
// └──────────────┘      │
//                       ▼
// ┌──────────────┐    ┌──────────────────────┐
// │ QImage B     │──→ │ 共享数据块           │
// │ d-pointer ───┤    │  refcount: 2         │
// └──────────────┘    │  pixels: [RGB...]    │
//                     └──────────────────────┘

B.scaled(241, 200);  // 读取操作，不触发 detach
                     // 直接从共享数据读取像素

A.setPixel(0,0, ...); // 写入操作，触发 detach
                      // 1. 分配新数据块给 A
                      // 2. 复制像素数据
                      // 3. 原数据块 refcount-- (变为1)
                      // 4. B 仍指向原数据块
```

问题：在多线程环境下，detach（写时复制）过程中的引用计数操作**不是原子性的**。当一个线程读取共享数据（如 scaled() 读取像素），另一个线程的操作触发 refcount 变更时，存在竞态窗口。

4.3 崩溃时序分析



4.4 三方竞争条件详解

崩溃由三个线程的并发操作共同导致：

线程	操作	对共享数据的影响
Video线程	持续创建新 QImage 并通过 SIG_sendVideoFrame 发送	新 QImage 的创建和旧 QImage 的销毁会改变共享数据的引用计数
Main线程	paintEvent 中 m_img.scaled() 读取像素数据	scaled() 是只读操作，但需要通过 d-pointer 访问共享数据
Worker线程	delete[] buf 执行堆操作	堆的 free 操作可能改变堆内存布局，加速共享数据块被回收

竞态窗口：当 Main 线程的 `m_img.scaled()` 正在通过 d-pointer 遍历像素数据时，如果 Video 线程的 QImage 析构将 `refcount` 减至 0 并释放了像素数据块，Main 线程将读取已释放内存 → 访问违规崩溃。

4.5 根因总结

```
根因: QImage 隐式共享在多线程环境下的非原子引用计数操作

触发条件:
  1. m_img = img (浅拷贝) 导致多个 QImage 共享像素数据
  2. paintEvent 的 scaled() 在 Main 线程读取共享数据
  3. Video 线程和 Worker 线程的并发操作改变了引用计数或堆状态

结果: scaled() 读取到已释放的像素数据 → 访问违规 → 进程崩溃
```

五、解决方案（含完整代码实现）

5.1 核心修复：QImage 深拷贝

文件：WeChat/usershow.cpp **函数**：UserShow::slot_setImage **修改类型**：核心修复

修改前（有 Bug）

```
void UserShow::slot_setImage(QImage &img)
{
    m_img = img;  // 浅拷贝：m_img 与 img 共享同一块像素数据
    m_lastTime = QTime::currentTime();
    this->update();
}
```

修改后（修复版）

```
void UserShow::slot_setImage(QImage &img)
{
    // [修复] 使用深拷贝而非浅拷贝，防止 paintEvent 中 scaled()
    //        访问已释放的共享像素数据
    // 原Bug: m_img = img 是浅拷贝（Qt隐式共享），m_img 与 img 共享同一块像素数据。
    //         slot_sendVideoFrame 返回后 img 被销毁，引用计数减少。
    //         paintEvent 中 m_img.scaled() 读取像素数据时，如果引用计数管理出现
    //         并发问题（video线程可能正在创建新的QImage影响共享数据），会导致读取
    //         已释放内存 → 崩溃。
    // 修复: m_img = img.copy() 创建深拷贝，m_img 拥有完全独立的像素数据缓冲区，
    //        不受其他 QImage 生命周期影响。
    m_img = img.copy();
```

```
    //每次获得画面 更新时间
    m_lastTime = QTime::currentTime();

    // [修复] 切换到 QImage 模式时隐藏 YuvGLWidget，防止 GL 渲染层覆盖
    if (m_glWidget) {
        m_glWidget->hide();
    }

    this->update();
}
```

修复原理

`img.copy()` 创建完全独立的像素数据缓冲区，`m_img` 不再与任何其他 QImage 共享数据。后续 `paintEvent` 中的 `m_img.scaled()` 读取的是独立的、不会被其他线程影响的数据。

```
修复前:  m_img ──┐
                 ├──→ [共享数据块 refcount=2] ← Video线程/Worker线程可影响
         img ────┘

修复后:  m_img ──→ [独立数据块 refcount=1] ← 仅 Main 线程可访问
         img ────→ [独立数据块 refcount=1] ← 独立管理
```

5.2 辅助修复：paintEvent 安全性增强

文件：WeChat/usershow.cpp **函数**：UserShow::paintEvent **修改类型**：防御性编程

完整修复后代码

```
void UserShow::paintEvent(QPaintEvent *event)
{
    QPainter painter( this );
    // 画黑色背景
    painter.setBrush( QBrush( Qt::black ) );
    painter.drawRect( 0, 0, this->width(), this->height() );

    // [修复] 当 YuvGLWidget 可见时（H264远程视频模式），跳过 QImage 绘制
    if (m_glWidget && m_glWidget->isVisible()) {
        painter.end();
        return;
    }

    if( m_img.size().height()<=0 ) return;

    // 画视频帧 — 等比例缩放
    // [修复] 添加 ui->lb_name 空指针检查和目标尺寸有效性检查
```

```
    int labelH = (ui->lb_name) ? ui->lb_name->height() : 20;
    int targetW = this->width();
    int targetH = this->height() - labelH;
    if (targetW <= 0 || targetH <= 0) {
        painter.end();
        return;
    }
    m_img = m_img.scaled(targetW, targetH, Qt::KeepAspectRatio);

    int x = this->width() - m_img.width();
    x = x/2;
    int y = this->height() - m_img.height() - labelH;
    y = labelH + y/2;
    QPixmap pix = QPixmap::fromImage( m_img );
    painter.drawPixmap( x , y , pix );
    painter.end();
}
```

安全性增强说明

增强项	原代码	修复后	原因
ui->lb_name 空指针检查	直接 ui->lb_name->height()	(ui->lb_name) ? ui->lb_name->height() : 20	防止 UI 文件变更后空指针崩溃
目标尺寸有效性检查	无	if (targetW <= 0 || targetH <= 0) return	防止窗口最小化时 0 尺寸传入 scaled()
labelH 复用	直接用 ui->lb_name->height()	局部变量 labelH 复用	减少重复调用，代码更清晰

5.3 辅助修复：H264 编码器帧缓冲区标准化

文件：WeChat/VideoRender/h264encoder.cpp **修改类型**：FFmpeg 标准化

5.3.1 帧缓冲区分配

```
// 初始化时设置帧参数
m_frame->format = AV_PIX_FMT_YUV420P;
m_frame->width  = width;
m_frame->height = height;

// [修复] 使用 av_frame_get_buffer 替代 av_image_alloc
// 原因: av_image_alloc 绕过 AVBufferRef 引用计数机制，
```

```
//       avcodec_send_frame 内部 av_frame_ref 只能浅拷贝指针，
//       导致编码器与外部帧共享缓冲区。
int ret = av_frame_get_buffer(m_frame, 0);
if (ret < 0) {
    qCritical() << "H264Encoder: av_frame_get_buffer failed, ret=" << ret;
    freeResources();
    return false;
}
```

5.3.2 帧可写性保证

```
// 每次 encode 前确保帧缓冲区可写
// 如果编码器仍持有上一帧的引用（引用计数 > 1），此函数会分配新缓冲区
// 如果引用计数 == 1（编码器已释放），直接复用现有缓冲区
int ret = av_frame_make_writable(m_frame);
if (ret < 0) {
    qWarning() << "H264Encoder::encode: av_frame_make_writable failed," << ret;
    return QByteArray();
}
```

5.3.3 RGB→YUV420P 转换

```
// [修复] 使用 FFmpeg sws_scale 替代手动 BT.601 转换
// 原因: 1. sws_scale 是 FFmpeg 标准函数，经过充分测试
//       2. 内部使用 SIMD 优化（SSE/AVX），性能更高
//       3. 边界检查完善，不会越界访问
// 前提: FFmpeg DLLs 是 MinGW 编译的（确认依赖 msvcrt.dll），
//       配合 .dll.a 导入库可以安全使用

// 初始化时创建转换器
m_swsCtx = sws_getContext(width, height, AV_PIX_FMT_RGB24,
                          width, height, AV_PIX_FMT_YUV420P,
                          SWS_BILINEAR, nullptr, nullptr, nullptr);

// encode 时执行转换
const uint8_t *srcData[1] = { rgbImg.constBits() };
int srcLinesize[1] = { rgbImg.bytesPerLine() };
sws_scale(m_swsCtx, srcData, srcLinesize, 0, m_height,
          m_frame->data, m_frame->linesize);
```

5.3.4 资源释放

```
void H264Encoder::freeResources()
{
    // 释放顺序: SwsContext → AVFrame → AVPacket → AVCodecContext
    // 注意: av_frame_free 会通过 AVBufferRef 自动释放缓冲区
    //       无需手动释放 m_yuvBuffer（已移除）
```

```
    if (m_swsCtx) {
        sws_freeContext(m_swsCtx);
        m_swsCtx = nullptr;
    }
    if (m_frame) {
        av_frame_free(&m_frame);
    }
    if (m_pkt) {
        av_packet_free(&m_pkt);
    }
    if (m_codecCtx) {
        avcodec_free_context(&m_codecCtx);
    }
    m_frameIndex = 0;
}
```

5.3.5 完整 encode 函数

```
QByteArray H264Encoder::encode(const QImage &img)
{
    if (!m_codecCtx || !m_frame) {
        qWarning() << "H264Encoder::encode: encoder not ready";
        return QByteArray();
    }

    // 验证编码器上下文完整性
    if (m_codecCtx->width != m_width || m_codecCtx->height != m_height
        || m_codecCtx->pix_fmt != AV_PIX_FMT_YUV420P) {
        qWarning() << "H264Encoder::encode: codecCtx corrupted";
        return QByteArray();
    }

    // 深拷贝 QImage，防止 use-after-free
    QImage rgbImg;
    if (img.format() == QImage::Format_RGB888) {
        rgbImg = img.copy();
    } else {
        rgbImg = img.convertToFormat(QImage::Format_RGB888);
    }

    // 防御性尺寸校验
    if (rgbImg.width() != m_width || rgbImg.height() != m_height) {
        rgbImg = rgbImg.scaled(m_width, m_height, Qt::IgnoreAspectRatio);
    }
    if (rgbImg.isNull() || !rgbImg.constBits()) {
        qWarning() << "H264Encoder::encode: rgbImg is null";
        return QByteArray();
    }

    // 确保帧缓冲区可写
```

```
    int ret = av_frame_make_writable(m_frame);
    if (ret < 0) return QByteArray();

    // RGB → YUV420P 转换
    const uint8_t *srcData[1] = { rgbImg.constBits() };
    int srcLinesize[1] = { rgbImg.bytesPerLine() };
    sws_scale(m_swsCtx, srcData, srcLinesize, 0, m_height,
              m_frame->data, m_frame->linesize);

    // 设置 PTS 和关键帧标志
    m_frame->pts = m_frameIndex++;
    m_frame->pict_type = m_requestKeyFrame ? AV_PICTURE_TYPE_I
                                           : AV_PICTURE_TYPE_NONE;
    m_requestKeyFrame = false;

    // 编码
    QByteArray result;
    ret = avcodec_send_frame(m_codecCtx, m_frame);
    if (ret < 0) return QByteArray();

    while (ret >= 0) {
        ret = avcodec_receive_packet(m_codecCtx, m_pkt);
        if (ret == AVERROR(EAGAIN) || ret == AVERROR_EOF) break;
        if (ret < 0) break;

        if (m_pkt->data && m_pkt->size > 0) {
            result.append(reinterpret_cast<const char*>(m_pkt->data),
                          m_pkt->size);
        }
        av_packet_unref(m_pkt);
    }
    return result;
}
```

5.4 辅助修复：导入库格式修正

文件：WeChat/vcc-client.pro、WeChat/VideoRender/videorender.pri **修改类型**：构建配置修正

vcc-client.pro 修改

```
# 修改前（MSVC 格式）
LIBS += $$PWD/ffmpeg-4.2.2/lib/libavcodec.lib \
        $$PWD/ffmpeg-4.2.2/lib/libavformat.lib \
        $$PWD/ffmpeg-4.2.2/lib/libavutil.lib \
        $$PWD/ffmpeg-4.2.2/lib/libswscale.lib

# 修改后（MinGW 标准格式）
```

```
LIBS += $$PWD/ffmpeg-4.2.2/lib/libavcodec.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavdevice.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavfilter.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavformat.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavutil.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libpostproc.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libswresample.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libswscale.dll.a
```

videorender.pri 修改

```
# 修改前
LIBS += $$FFMPEG_PATH/lib/libavcodec.lib \
        $$FFMPEG_PATH/lib/libavutil.lib \
        $$FFMPEG_PATH/lib/libswscale.lib

# 修改后
LIBS += $$FFMPEG_PATH/lib/libavcodec.dll.a \
        $$FFMPEG_PATH/lib/libavutil.dll.a \
        $$FFMPEG_PATH/lib/libswscale.dll.a
```

修正原因

导入库格式	编译器	说明
`.lib`	MSVC	COFF 格式，符号修饰与 MinGW 不完全匹配
`.dll.a`	MinGW	标准 GNU 格式，符号修饰完全匹配

虽然 `.lib` 在多数情况下可以工作（MinGW 的 `ld` 链接器能解析 COFF 格式），但符号修饰的细微差异可能导致函数指针解析异常、虚表布局不匹配等隐蔽问题。使用 `.dll.a` 彻底消除此风险。

5.5 辅助修复：编码器重初始化方式

文件：`WeChat/ckernel.cpp` **函数**：`ckernel::slot_startVideo` **修改类型**：防止 FFmpeg 内部状态残留

```
// 原代码（close+init 在同一对象上重初始化）
m_h264Encoder->close();
bool initOk = m_h264Encoder->init(encWidth, encHeight, encFps, encBitrate);

// 修复后（销毁旧实例，创建全新实例）
delete m_h264Encoder;
```

```
m_h264Encoder = new H264Encoder(this);
bool initOk = m_h264Encoder->init(encWidth, encHeight, encFps, encBitrate);
```

修正原因

close() 释放 codecCtx 后，FFmpeg 全局状态可能被污染。在同一对象上调用 init() 创建的新 SwsContext 在 sws_scale 时可能访问损坏内存。使用 delete + new 创建全新实例，确保无残留状态。

5.6 辅助修复：Lambda 捕获 QImage 深拷贝

文件：WeChat/ckernel.cpp **函数**：ckernel::slot_sendVideoFrame **修改类型**：防止 use-after-free

```
// 原代码（浅拷贝捕获，lambda 执行时像素数据可能已释放）
QTimer::singleShot(0, this, [this, img]() {
    encodeAndSendVideoFrame(img);
});

// 修复后（深拷贝捕获，独立像素数据）
QImage imgCopy = img.copy();  // 深拷贝，独立像素数据
QTimer::singleShot(0, this, [this, imgCopy]() {
    encodeAndSendVideoFrame(imgCopy);
    m_encodingPending = false;
});
```

修正原因

[this, img] 捕获 img 是浅拷贝（Qt 隐式共享）。lambda 被投递到事件队列后，摄像头线程可能在 lambda 执行前释放或修改 img 的底层像素数据，导致 encode() 中访问已释放内存。

六、修改文件清单与差异对比

6.1 修改文件总览

文件	修改类型	修改项数	说明
WeChat/usershow.cpp	**核心修复**	2	slot_setImage 深拷贝 + paintEvent 安全性

文件	修改类型	修改项数	说明
WeChat/VideoRender/h264encoder.cpp	辅助修复	4	帧缓冲区 + sws_scale + 资源释放 + 编码流程
WeChat/ckernel.cpp	辅助修复	3	编码器重建 + Lambda深拷贝 + 清理调试代码
WeChat/vcc-client.pro	辅助修复	1	导入库 .lib → .dll.a
WeChat/VideoRender/videorender.pri	辅助修复	1	导入库 .lib → .dll.a
WeChat/VideoRender/h264decoder.cpp	清理	1	移除未使用的 csignal/csetjmp 头文件

6.2 核心差异对比

usershow.cpp — slot_setImage

```
 void UserShow::slot_setImage(QImage &img)
 {
-    m_img = img;
+    m_img = img.copy();
     m_lastTime = QTime::currentTime();
+    if (m_glWidget) {
+        m_glWidget->hide();
+    }
     this->update();
 }
```

usershow.cpp — paintEvent

```
 void UserShow::paintEvent(QPaintEvent *event)
 {
     QPainter painter( this );
     painter.setBrush( QBrush( Qt::black ) );
     painter.drawRect( 0, 0, this->width(), this->height() );

+    if (m_glWidget && m_glWidget->isVisible()) {
+        painter.end();
+        return;
+    }
+
     if( m_img.size().height()<=0 ) return;
```

```
-    m_img = m_img.scaled( this->width() , this->height() - ui->lb_name->height(),
-                          Qt::KeepAspectRatio );
+    int labelH = (ui->lb_name) ? ui->lb_name->height() : 20;
+    int targetW = this->width();
+    int targetH = this->height() - labelH;
+    if (targetW <= 0 || targetH <= 0) {
+        painter.end();
+        return;
+    }
+    m_img = m_img.scaled(targetW, targetH, Qt::KeepAspectRatio);

     int x = this->width() - m_img.width();
     x = x/2;
-    int y = this->height() - m_img.height() - ui->lb_name->height();
-    y = ui->lb_name->height() + y/2;
+    int y = this->height() - m_img.height() - labelH;
+    y = labelH + y/2;
     QPixmap pix = QPixmap::fromImage( m_img );
     painter.drawPixmap( x , y , pix );
     painter.end();
 }
```

h264encoder.cpp — init (帧缓冲区)

```
-    // RGB->YUV420P 转换器（当前未使用，保留接口）
-    m_swsCtx = nullptr;
+    // 创建 RGB->YUV420P 转换器 (SwsContext)
+    m_swsCtx = sws_getContext(width, height, AV_PIX_FMT_RGB24,
+                              width, height, AV_PIX_FMT_YUV420P,
+                              SWS_BILINEAR, nullptr, nullptr, nullptr);
+    if (!m_swsCtx) {
+        freeResources();
+        return false;
+    }

-    // 使用 av_image_alloc 手动分配
-    int bufSize = av_image_get_buffer_size(AV_PIX_FMT_YUV420P, width, height, 1);
-    m_yuvBuffer = (uint8_t*)av_malloc(bufSize);
-    av_image_fill_arrays(m_frame->data, m_frame->linesize, m_yuvBuffer, ...);
+    // 使用标准的 av_frame_get_buffer 分配
+    int ret = av_frame_get_buffer(m_frame, 0);
```

h264encoder.cpp — encode (转换方式)

```
-    // 手动 BT.601 RGB→YUV 转换
-    for (int y = 0; y < m_height; y++) {
-        for (int x = 0; x < m_width; x++) {
-            int yVal = ((66 * r + 129 * g + 25 * b + 128) >> 8) + 16;
```

```
-            yPlane[y * yStride + x] = ...;
-            if ((y & 1) == 0 && (x & 1) == 0) {
-                uPlane[...] = ...;
-                vPlane[...] = ...;
-            }
-        }
-    }
+    // 使用 FFmpeg sws_scale
+    const uint8_t *srcData[1] = { rgbImg.constBits() };
+    int srcLinesize[1] = { rgbImg.bytesPerLine() };
+    sws_scale(m_swsCtx, srcData, srcLinesize, 0, m_height,
+              m_frame->data, m_frame->linesize);
```

七、测试验证

7.1 验证方法

1. 使用 Qt 5.12.11 (MinGW 7.3.0 32-bit) 编译 Release 版本
2. 连接服务器（192.0.2.130:8000）
3. 登录并启动视频通话
4. 观察程序是否持续运行（超过 30 秒无崩溃）
5. 观察视频画面是否正常显示
6. 观察日志中是否出现多帧 `slot_sendVideoFrame called`（视频循环正常）

7.2 验证结果

测试项	预期结果	实际结果	状态
程序启动	正常启动，无崩溃	正常启动	✅ 通过
H264 编码器初始化	libx264 编码器成功加载	libx264 成功	✅ 通过
第一帧编码	编码成功，无崩溃	编码成功	✅ 通过
视频帧循环	持续编码发送多帧	持续循环	✅ 通过
程序稳定性	30 秒以上无崩溃	稳定运行	✅ 通过
视频画面显示	本地预览正常显示	正常显示	✅ 通过
代码整洁性	无调试代码残留	已清理	✅ 通过

7.3 编译验证

```
构建环境: Qt 5.12.11 + MinGW 7.3.0 32-bit
构建配置: Release
```

```
编译结果: BUILD OK (0 errors, 0 warnings)
输出文件: WeChat.exe (4,475,392 bytes)
```

7.4 运行日志验证

修复后的正常日志时序:

```
H264Encoder: init success, 640 x 480 @ 30 fps encoder: "libx264"
=== 视频启动完成, 编码模式: H264 ===

slot_sendVideoFrame called, img size: 640 x 480 , m_id: 3     ← 第1帧
slot_sendVideoFrame called, img size: 640 x 480 , m_id: 3     ← 第2帧（修复前在此崩溃）
slot_sendVideoFrame called, img size: 640 x 480 , m_id: 3     ← 第3帧
...                                                             ← 持续运行
slot_sendVideoFrame called, img size: 640 x 480 , m_id: 3     ← 第N帧
```

八、调试代码清理记录

8.1 清理原则

修复过程中添加的所有调试代码已**全部移除**，保留的日志仅限于:

- 编码器生命周期日志（init/close）— 仅在启停时触发一次
- 错误警告日志（qWarning/qCritical）— 仅在异常时触发
- 分片发送摘要日志 — 每帧一条，用于生产环境监控

8.2 清理清单

调试代码	文件	位置	清理状态
`#include <windows.h>`	h264encoder.cpp	第4-6行	✅ 已移除
`HeapValidate` 编码前检查	h264encoder.cpp	encode() 内	✅ 已移除
`HeapValidate` 编码后检查	h264encoder.cpp	encode() 内	✅ 已移除
心跳定时器	ckernel.cpp	构造函数	✅ 已移除
`[heartbeat] main thread alive`	ckernel.cpp	心跳回调	✅ 已移除

调试代码	文件	位置	清理状态
`[paintEvent] enter/scaled/done`	usershow.cpp	paintEvent	✅ 已移除（8处）
`[lambda] encodeAndSendVideoFrame done`	ckernel.cpp	lambda 内	✅ 已移除
`[encodeAndSendVideoFrame] returning`	ckernel.cpp	函数末尾	✅ 已移除
`[slot_SendVideo] before/after delete[]`	ckernel.cpp	slot_SendVideo	✅ 已移除
`=== encodeAndSendVideoFrame enter ===`	ckernel.cpp	函数入口	✅ 已移除
`=== calling m_h264Encoder->encode ===`	ckernel.cpp	编码前	✅ 已移除
`=== encode returned ===`	ckernel.cpp	编码后	✅ 已移除
`#include <csignal>`	h264decoder.cpp	头文件区	✅ 已移除
`#include <csetjmp>`	h264decoder.cpp	头文件区	✅ 已移除
`H264Encoder::encode` 入口日志	h264encoder.cpp	encode() 首行	✅ 已移除

8.3 保留的生产日志

日志	文件	触发频率	保留原因
`H264Encoder: init success`	h264encoder.cpp	启动时1次	编码器初始化诊断
`H264Encoder: using encoder`	h264encoder.cpp	启动时1次	确认实际使用的编码器
`=== H264编码器激活流程开始 ===`	ckernel.cpp	启动时1次	编码流程入口标记

日志	文件	触发频率	保留原因
`=== 视频启动完成 ===`	ckernel.cpp	启动时1次	编码模式确认
`[slot_sendVideoFrame] H264 encode:`	ckernel.cpp	每帧1条	帧编码摘要监控
`[slot_SendVideo] SendData OK`	ckernel.cpp	每分片1条	网络发送监控

九、经验总结与最佳实践

9.1 Qt 隐式共享的线程安全陷阱

陷阱说明

Qt 的隐式共享机制在**单线程**环境下是安全高效的，但在**多线程**环境下存在陷阱：

- QImage 的赋值 (=) 是浅拷贝，多个 QImage 对象共享同一块像素数据
- 引用计数管理在 detach（写时复制）时存在竞态窗口
- 当一个线程读取共享数据（如 scaled()），另一个线程修改引用计数时，可能导致数据被提前释放

最佳实践

```
// ❌ 错误：跨线程传递 QImage 使用浅拷贝
void slot_receiveImage(QImage img) {
    m_img = img;  // 浅拷贝，多线程下不安全
}

// ✅ 正确：跨线程传递 QImage 使用深拷贝
void slot_receiveImage(QImage img) {
    m_img = img.copy();  // 深拷贝，独立像素数据
}

// ✅ 正确：Lambda 捕获 QImage 使用深拷贝
QImage imgCopy = img.copy();
QTimer::singleShot(0, this, [this, imgCopy]() {
    processImage(imgCopy);
});
```

适用范围

以下 Qt 类型使用隐式共享，跨线程传递时均需注意：

- `QImage`、`QPixmap`
- `QByteArray`、`QString`
- `QList`、`QVector`、`QMap`
- `QPen`、`QBrush`、`QFont`

9.2 崩溃诊断方法论

有效的诊断手段（按优先级排列）

优先级	方法	适用场景	效果
1	**逐步插桩法**	崩溃位置未知	在关键路径添加日志，逐步缩小范围
2	**堆完整性检查**	怀疑堆腐蚀	`HeapValidate` 确认堆状态
3	**心跳监测法**	怀疑线程死锁/崩溃	定时器监测线程存活状态
4	**并发分析**	多线程竞争条件	分析线程时序，识别竞态窗口

无效的诊断方向（本次走过的弯路）

弯路	耗时	教训
误判 FFmpeg DLL 为 MSVC 编译	~1h	应先检查 DLL 依赖（`msvcrt.dll` vs `msvcrXXX.dll`）
过度关注 FFmpeg ABI 兼容性	~2h	`HeapValidate` 通过应排除堆腐蚀假设
`signal(SIGSEGV)` + `setjmp/longjmp`	~1h	不跨 DLL 边界，且腐蚀 SEH 链导致更难诊断
手动 RGB→YUV 转换替代 sws_scale	~1h	应先确认 DLL 编译工具链再决定是否能用

9.3 FFmpeg 编码器使用最佳实践

标准帧管理流程

```
// 1. 初始化
m_frame = av_frame_alloc();
m_frame->format = AV_PIX_FMT_YUV420P;
m_frame->width  = width;
```

```
m_frame->height = height;
av_frame_get_buffer(m_frame, 0);  // 标准分配，创建 AVBufferRef

// 2. 每帧编码前
av_frame_make_writable(m_frame);  // 确保可写（引用计数 > 1 时自动分配新缓冲区）

// 3. 填充帧数据
sws_scale(m_swsCtx, ...);  // 使用 sws_scale 进行色彩空间转换

// 4. 编码
avcodec_send_frame(m_codecCtx, m_frame);
while (avcodec_receive_packet(m_codecCtx, m_pkt) >= 0) {
    // 处理编码数据
    av_packet_unref(m_pkt);
}

// 5. 释放
av_frame_free(&m_frame);      // 自动释放缓冲区（通过 AVBufferRef）
av_packet_free(&m_pkt);
avcodec_free_context(&m_codecCtx);
sws_freeContext(m_swsCtx);
```

禁止事项

```
// ❌ 禁止：使用 av_image_alloc 绕过 AVBufferRef
uint8_t *buffer = (uint8_t*)av_malloc(bufSize);
av_image_fill_arrays(m_frame->data, m_frame->linesize, buffer, ...);

// ❌ 禁止：不调用 av_frame_make_writable 直接写入 data[]
m_frame->data[0][0] = 255;  // 编码器可能仍持有引用

// ❌ 禁止：在同一对象上 close()+init() 重初始化
encoder->close();
encoder->init(...);  // FFmpeg 内部状态可能残留

// ✅ 正确：使用 delete+new 创建全新实例
delete encoder;
encoder = new H264Encoder(this);
encoder->init(...);
```

9.4 调试代码管理规范

原则

1. **调试代码必须标记**：使用统一的注释前缀（如 [诊断]）便于后续清理
2. **调试代码必须集中**：不要分散在业务逻辑中，尽量使用独立的代码块
3. **修复完成后立即清理**：调试代码不应进入版本控制

4. **保留的日志必须有价值**：仅保留生命周期关键节点和错误路径的日志

清理检查清单

- ☐ 所有 [诊断] 标记的代码已移除
- ☐ 所有临时 #include 已移除（如 windows.h、csignal）
- ☐ 所有心跳定时器已移除
- ☐ 所有 qDebug 调试输出已移除或降级为生产日志
- ☐ 所有 HeapValidate 等运行时检查已移除
- ☐ 编译无新 warning
- ☐ 程序运行行为与清理前一致

十、附录

10.1 关键文件路径

文件	绝对路径
H264 编码器实现	WeChat\VideoRender\h264encoder.cpp
H264 编码器头文件	WeChat\VideoRender\h264encoder.h
H264 解码器实现	WeChat\VideoRender\h264decoder.cpp
YUV OpenGL 渲染	WeChat\VideoRender\yuvglwidget.cpp
核心内核	WeChat\ckernel.cpp
用户视频显示	WeChat\usershow.cpp
视频采集	WeChat\VideoApi\videoread.cpp
项目配置	WeChat\vcc-client.pro
VideoRender 配置	WeChat\VideoRender\videorender.pri

10.2 FFmpeg DLL 依赖确认

```
FFmpeg 版本: 4.2.2 (Zeranoe 构建)
编译器: MinGW (依赖 msvcrt.dll，非 MSVC)
导入库: .dll.a (MinGW 标准格式)
编码器: libx264 (软编码)
  - h264_nvenc: 不可用 (无 NVIDIA GPU)
  - h264_qsv: 不可用 (无 Intel QSV)
  - libx264: 可用 (软件编码)
```

10.3 构建配置

```
# vcc-client.pro — FFmpeg 库引用
INCLUDEPATH += $$PWD/ffmpeg-4.2.2/include
LIBS += $$PWD/ffmpeg-4.2.2/lib/libavcodec.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavdevice.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavfilter.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavformat.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavutil.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libpostproc.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libswresample.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libswscale.dll.a
```

10.4 编码参数配置

参数	值	说明
分辨率	640 x 480	SD 标清
帧率	30 fps	实时通信标准帧率
码率	1,200,000 (1.2 Mbps)	适配 640x480 分辨率
GOP	60 (2秒)	2 秒一个 I 帧
B 帧	0 (禁用)	实时通信不需要 B 帧
preset	veryfast	编码速度优先
tune	zerolatency	零延迟模式
CRF	23	恒定质量因子
profile	main	主流兼容性
像素格式	YUV420P	广泛兼容
分片大小	4096 字节	适配 UDP/TCP 传输

10.5 术语表

术语	说明
隐式共享	Qt 的 Copy-On-Write 机制，多个对象共享同一数据，修改时才复制
detach	隐式共享中，修改共享数据时触发的实际复制操作
AVBufferRef	FFmpeg 的引用计数缓冲区管理结构

术语	说明
sws_scale	FFmpeg 的图像缩放和色彩空间转换函数
av_frame_make_writable	确保帧缓冲区可写，引用计数 > 1 时自动分配新缓冲区
HeapValidate	Windows API，验证进程堆的完整性
PLI	Picture Loss Indication，WebRTC 中的关键帧请求机制
GOP	Group of Pictures，一个 I 帧到下一个 I 帧之间的帧组